Collection & Classification

In research

SLR (Systematic Literature Review) vs. Empirical experiment / Comparative analysis

Improving Software Defect Prediction Using Machine Learning Models Compared with Traditional Rule-Based Approaches

- RQ1: How are traditional rule-based defect prediction methods applied in software defect prediction?
- RQ2: Which approach is more suitable for software quality improvement when considering the trade-offs between prediction performance, interpretability, implementation complexity, and practical usability?

SLR vs. Empirical experiment / Comparative analysis

- Eg. Comparative Study of GenAI-Generated and Human-Authored Black-Box Test Suites
- SLR involvement:
  - LLM-generated test cases;
  - black-box testing;
  - requirement-based test generation;
  - AI vs human test suite comparison;
  - coverage, mutation score, fault detection, semantic correctness;
  - human-AI collaboration in testing.
- SLR output should be synthesis, such as:
  - GenAI often improves breadth of requirement/viewpoint coverage;
  - Human testers are usually stronger in domain reasoning, boundary cases, and semantic correctness;
  - Existing studies often lack consistent defect-detection metrics;
  - Few studies compare GenAI and human-authored black-box tests under the same criteria.

6. Literature Matrix

Author	Method	Dataset	Metrics	Findings	Limitations
Menzies et al. (2007)	Data Mining Techniques	NASA	Accuracy	Machine learning techniques improved software defect prediction compared with simple threshold-based methods.	The study was conducted using a limited range of datasets, which may affect the generalizability of the results.
Lessmann et al. (2008)	Random Forest, SVM, Logistic Regression	PROMISE Datasets	AUC	Random Forest demonstrated strong defect prediction	The study focused primarily on comparing machine learning

PRISMA

- Preferred Reporting Items for Systematic Reviews and Meta-Analyses.
- a reporting framework/checklist that helps researchers show
  - **how they searched,**
  - **selected,**
  - **excluded,**
  - **and analyzed papers**.
- is commonly used to make the literature review transparent and reproducible.
- is the process for showing the paper-selection pipeline.

PRISMA in practice

Eg. Comparative Study of GenAI-Generated and Human-Authored Black-Box Test Suites

- Identification
  - Search
- Screen
  - Remove duplicate papers
  - Read titles and abstracts
  - Exclude papers unrelated to software testing or not about GenAI/LLMs
- Eligibility
  - Read full papers.
    Keep only studies that have empirical evidence, metrics, datasets, or test-generation experiments.
- Included Studies
  - Final selected papers become the evidence base for the SLR

PRISMA in practice

Stage	No. of documents
Found from searching	100
After removing duplicates	80
After title/abstract screening	30
After full-text eligibility check	15
Final studies included	10

Include Criteria:

- GenAI/LLMs for software test generation;
- involve black-box, requirement-based, or functional test cases;
- empirical results;
- metrics: coverage, mutation score, fault detection, correctness, or human comparison;
- newly published
- available in full text

Exclude Criteria:

- no experiment;
- duplicate;
- not software testing;
- not enough methodological detail;
- inaccessible

Literature Matrix

- Data extraction from sources:
  - For each selected paper, students should extract the same fields.

Field	Example
Paper title	
Published Year	
Metrics	
Main finding	
Limitations	
Dataset	
Methods	
Human baselined?	Yes / No
…	
Relevance to this research	High / Medium / Low

- Compare author, method, dataset, metrics, findings, limitations

Quality assessment

- Should score each study
  - Using checklist, such as

Question	Score
research questions clearly stated?	0/1
dataset described?	0/1
metrics clearly defined?	
are threats to validity discussed?	
human baselined?	

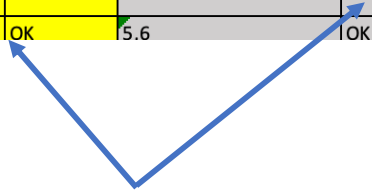
Measurement

- For empirical part, need to define the way to measure, such as

Measurement	Formula
Viewpoint coverage	Covered viewpoints / total viewpoints
Category coverage	Covered viewpoints in category / total viewpoints in category
Missing rate	Missed viewpoints / total viewpoints
Unique contribution	Viewpoints covered only by one model
Overlap with human tests	AI-human shared viewpoints / total viewpoints
Human-AI combined coverage	Union of human + AI covered viewpoints / total viewpoints
…	

Measurement

B	C	D	E	F	G	H
Test Viewpoint	**Num. Test cases**	**Sonnet 4.6**	**Num. Test cases**	**Gemini 3.1**	**Num. Test cases**	**ChatGPT 5.5**
Successfully add a task with required fields only	2, 32, 33,40,41,43,57,58	OK	1,5,6,10,11,14,,47,49	OK	1,4,34,35,38,44,46,,61,62	OK
Successfully add a task with explicit low, medium, and high priority	1, 3	OK	2	OK	2,3,24	OK
Verify default priority is medium when --priority is omitted	2	OK	1	OK	1	OK
Automatically assign task IDs starting from 1 in creation order	3	OK	1,30	OK	4	OK
Update only the title of an existing task	10,14	OK	18	OK	17	OK
Update only the due date of an existing task	11	OK			18,22	OK
Update only the priority of an existing task	12	OK	22	OK	19	OK
Update multiple editable fields in one command	13	OK	19	OK	20,21	OK
Update a completed task without changing its done status	14,60	OK	22	OK	21,22	OK
Complete a todo task and reflect the done status in list output	4	OK	23	OK	10	OK
Reopen a done task and reflect the todo status in list output	5	OK	26	OK	12	OK
Delete todo and done tasks and verify they disappear from list and summary	6,23	OK	28	OK	13	OK
Verify deleted task IDs are not reused after adding a new task	9,57	OK	30	OK	16,62	OK
List all tasks, only todo tasks, and only done tasks	16, 17, 18	OK	32	OK	7,8,9,10	OK
Sort tasks by created order, due date, and priority severity			36	OK	7,23	OK
Combine status filtering with sorting	59	OK			26	OK
Show summary counts after add, complete, reopen, and delete operations	23,24,60		42	OK	11,27	OK
Reject missing command, unknown command, and unknown option	51,52,53	OK		OK	34	OK
Reject blank title, 0-character title, and title longer than 80 characters	32,35,34				28	OK
Accept minimum and maximum valid title length: 1 and 80 characters	32,33	OK	5,6	OK	34,35	OK



Ok if has at least one test case for this Test viewpoint.
Otherwise, leave it blank

Seeded Faults / Mutants

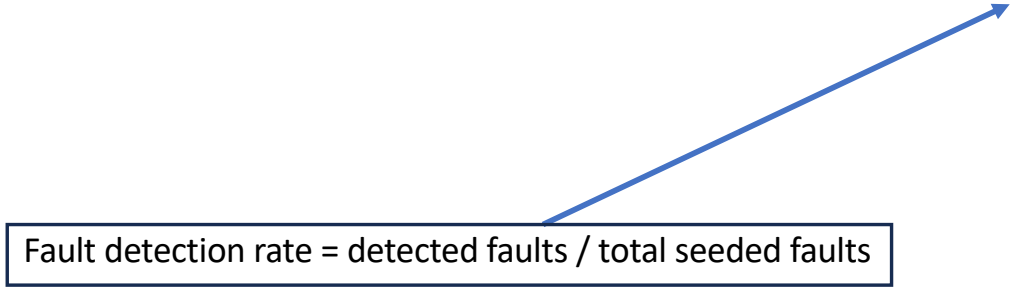
- A **seeded fault** is an intentional bug inserted into the application.
- A **mutant** is a small changed version of the program created by modifying one behavior, condition, formula, or rule.
- They are used to test whether a test suite can actually detect bugs.
- Answer the question: Can the suite catch a wrong implementation?

vs. coverage: answer the question: Did the suite mention this viewpoint?

Seeded Faults / Mutants

- **Task Manager:** Suppose correct rules include:
  - add task with title;
  - default priority is medium;
  - valid priorities are low, medium, high;
  - update title/due date/priority separately;
  - mark task as completed;
  - delete existing task;
  - reject invalid task ID.

Mutant	Bug introduced	Test needed to kill it
MUT01	Default priority becomes low instead of medium	Add task without priority
MUT02	Invalid priority is accepted	Add task with priority “urgent”
MUT03	Updating due date also changes title	Update only due date
MUT04	Complete command marks wrong task ID	Complete task when multiple tasks exist
MUT05	App allows empty title	Add task with empty title
MUT06	ask IDs start from 0 instead of 1	Add first task and check ID



If a test suite fails when run against mutant MUTX, it has **killed** that mutant.
- “Killed” means the test detected the bug.
- “Survived” means the faulty program still passed the tests.

Mutation score = killed mutants / total mutants

Seeded Faults / Mutants

```
def calculate_fine_correct(item_type, days_overdue, is_member=False, is_lost=False, replacement_fee=0):
    if days_overdue < 0:
        raise ValueError("days_overdue cannot be negative")

    if is_lost:
        return replacement_fee

    if item_type == "book":
        fine = min(days_overdue * 0.50, 20.00)
    elif item_type == "dvd":
        fine = min(days_overdue * 1.00, 30.00)
    else:
        raise ValueError("invalid item_type")

    if is_member:
        fine = fine * 0.50

    return round(fine, 2)
```

Correct program

Rules:

- Book fine: $0.50 per overdue day, capped at $20
- DVD fine: $1.00 per overdue day, capped at $30
- Members get 50% discount on overdue fines
- Lost items use replacement fee only
- Member discount must NOT apply to lost-item replacement fee
- Days overdue cannot be negative

```
def calculate_fine_m2_book_cap_wrong(item_type, days_overdue, is_member=False, is_lost=False, replacement_fee=0):
    if days_overdue < 0:
        raise ValueError("days_overdue cannot be negative")

    if is_lost:
        return replacement_fee

    if item_type == "book":
        fine = min(days_overdue * 0.50, 25.00)  # BUG: should cap at 20.00
    elif item_type == "dvd":
        fine = min(days_overdue * 1.00, 30.00)
    else:
        raise ValueError("invalid item_type")

    if is_member:
        fine = fine * 0.50

    return round(fine, 2)
```

Faulty version (mutant)

Each mutant contains exactly one intentional bug.

Test:

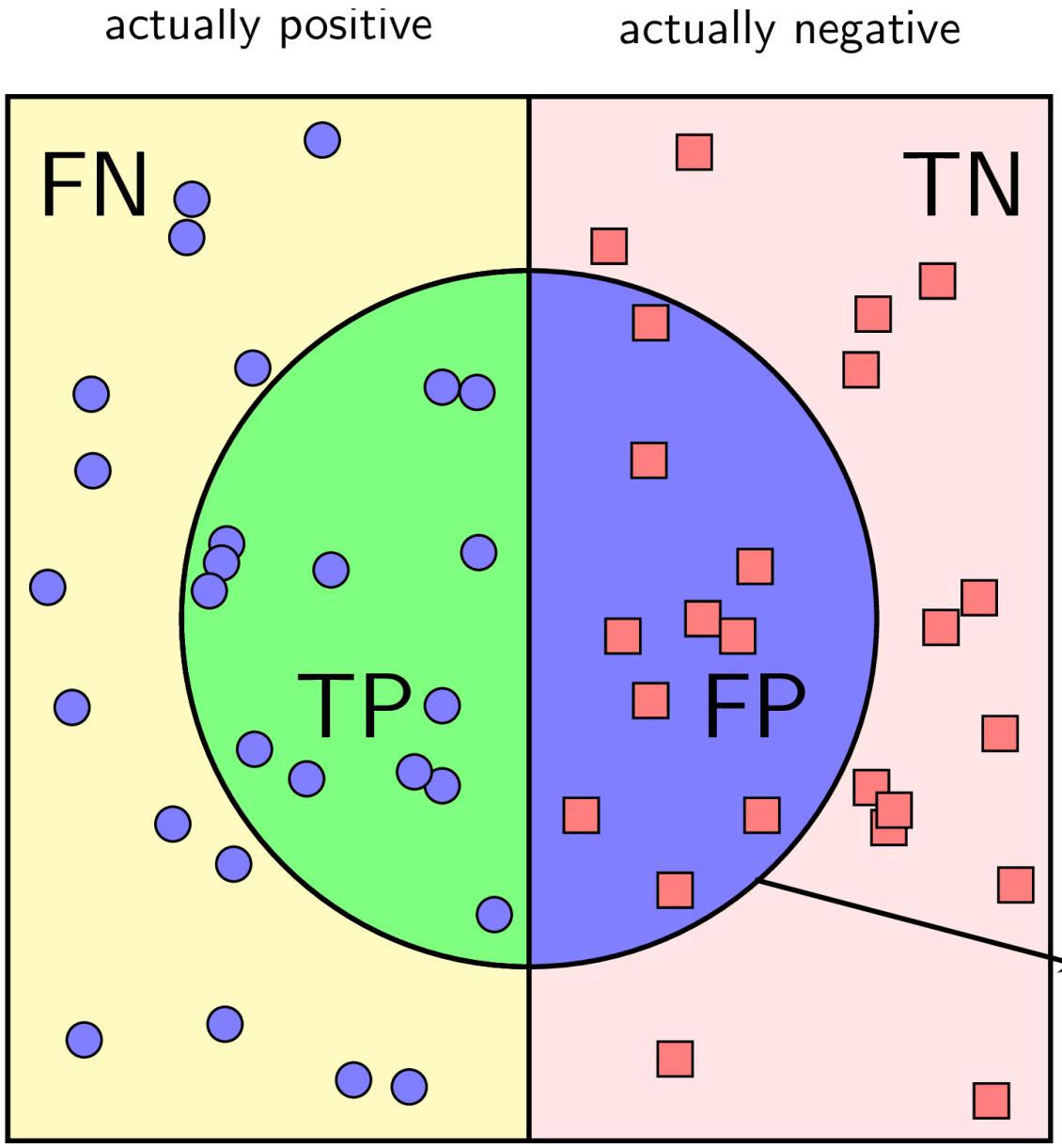
```
assert calculate_fine_m2_book_cap_wrong("book", 100) == 20.00
```

Assessment - classification performance metrics

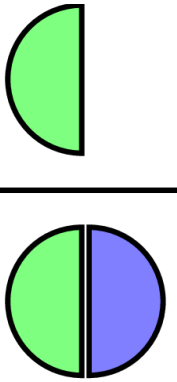
Metrics	Definition	Range	Better
TPR	$\frac{TP}{TP+FN}$	[0,1]	High
FPR	$\frac{FP}{TN+FP}$	[0,1]	Low
Precision	$\frac{TP}{TP+FP}$	[0,1]	High
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	[0,1]	High
F1	$\frac{2\times TPR\times Precision}{TPR+Precision}$	[0,1]	High
MCC	$\frac{TP\times TN-FP\times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$	[-1,1]	High
GM	$\sqrt{TPR\times(1-FPR)}$	[0,1]	High
AUC	Area Under Curve (AUC) ROC chart	[0,1]	High

True positive rate (TPR), also referred to as sensitivity or recall is which is the proportion of positive cases that are correctly considered as positive (defect-prone) as a proportion of all positive cases.

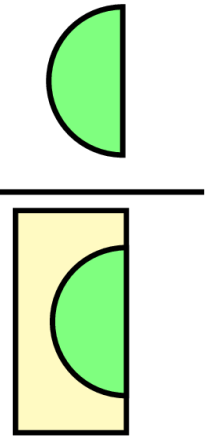
False positive rate (FPR) is the probability that a test incorrectly flags a negative condition as positive



$$\text{Precision} = \frac{\text{TP}}{\text{TP+FP}} =$$



$$\text{Recall} = \frac{\text{TP}}{\text{TP+FN}} =$$



classified (or found) as positive

G-Mean

F1 combines Precision and Recall/TPR.
balances ability to detect faulty cases and correctly pass non-faulty cases.
is useful when faulty and non-faulty samples are imbalanced.

Mathews Correlation Coefficient

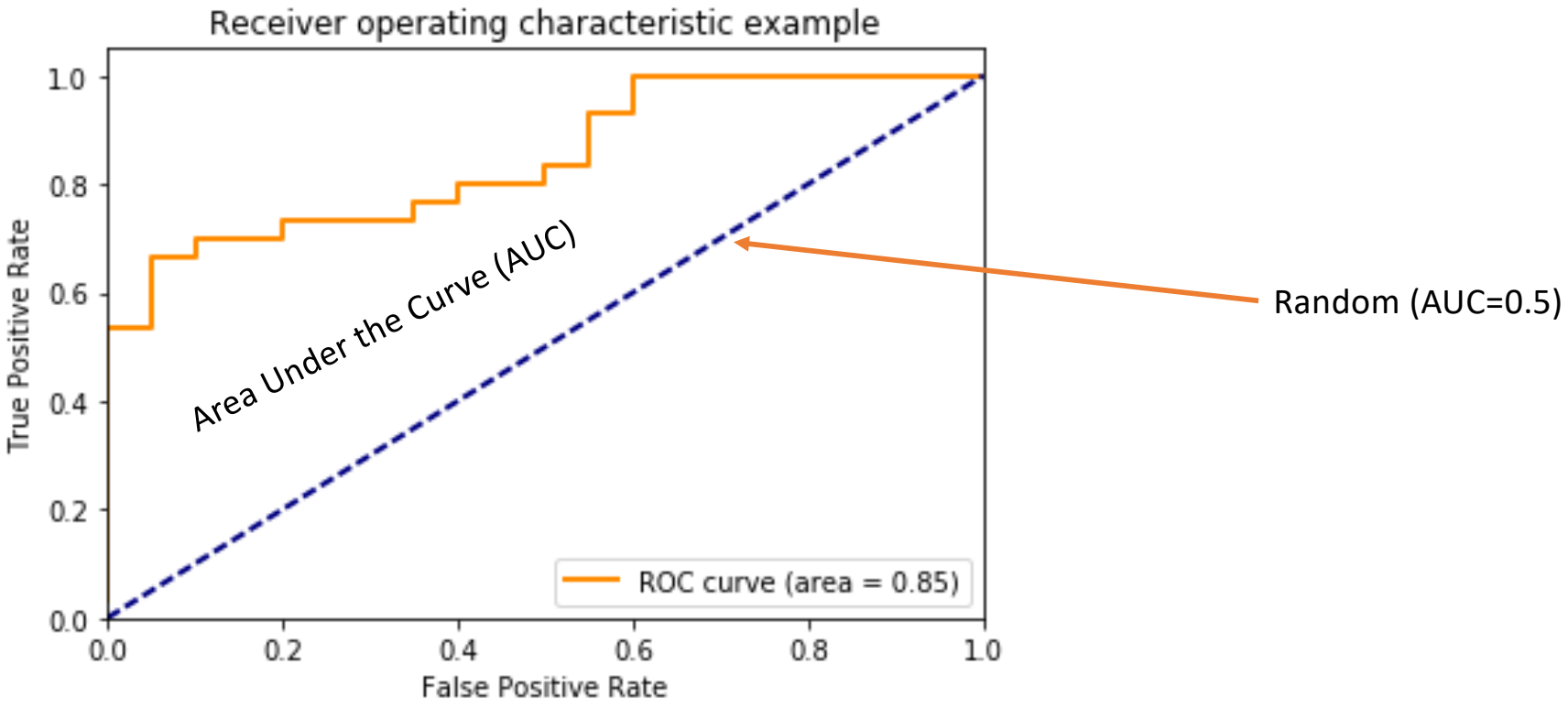
a strong balanced metric for binary classification.

MCC value	Explaination
+1	Perfect detection
0	Equal to random
-1	Completely wrong

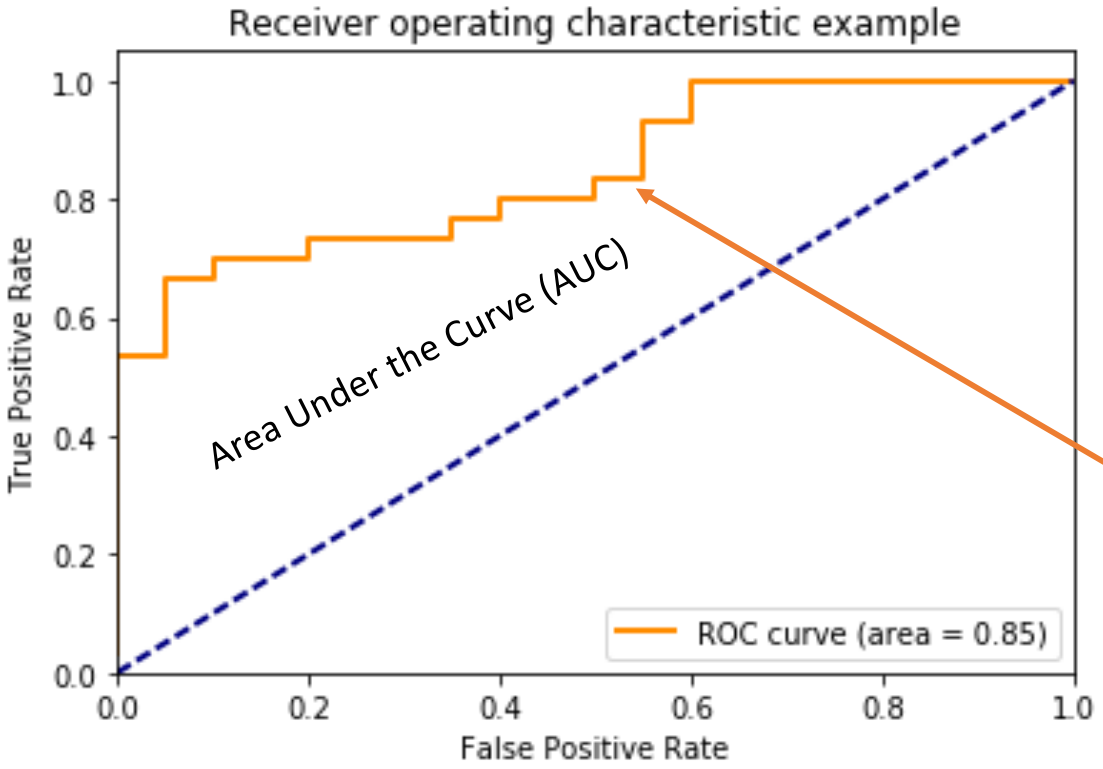
F1 score

balances ability to detect faulty cases and correctly pass non-faulty cases.

ROC - Receiver Operating Characteristic



ROC - Receiver Operating Characteristic



score = number of failed test cases against a program version

Threshold	Prediction Rule
>= 1 failed test	classify as faulty
>= 2 failed test	classify as faulty
>= 3 failed test	classify as faulty
..	

Each threshold gives a different TPR/FPR pair.

Missing Empirical Experiments

- Eg. Predictive Performance Evaluation of Artificial Intelligence Models in Software Defect Prediction: An Empirical Comparison of Machine Learning and Tabular Deep Learning

TABLE I
COMPARISON OF RELATED WORKS IN SOFTWARE DEFECT PREDICTION

#	Paper (Year)	AI Method	Dataset(s)	Metrics	Best Result	Key Limitations
1	Wang et al. (2020) [3]	DBN, RF, LR	PROMISE, NASA-MDP	F1-Score	F1 Improv +41.9%	Requires parsing AST token vectors; computationally expensive representation.
2	Menzies et al. (2007) [1]	RF, Naive Bayes, SVM	NASA-MDP, Jureczko	AUC, Precision	AUC = 0.78	Confined strictly to traditional ML baselines; no deep architectures evaluated.
3	Lessmann et al. (2008) [2]	RF, SVM, LR	NASA-MDP, Jureczko	AUC-ROC	RF is stable	Benchmarked only classical ML models; did not evaluate deep learning architectures.
4	Asal & Yalçıner (2026) [5]	TabNet, NODE, FT-Trans	NASA JM1 (PROMISE)	Precision, Acc	Precision = 99%	Evaluated on a single project; relies heavily on NearMiss undersampling.
5	Goyal (2026) [6]	DN_SDP (DL)	11 PROMISE datasets	MCC, F-measure	MCC Improv +42%	High architectural complexity; evaluated on small-scale tabular attributes.
6	Fei et al. (2025) [7]	TabNet + CNN-BiLSTM	10 PROMISE projects	F1-Score, AUC	F1 improved	High training overhead due to heterogeneous hybrid inputs.
7	Giray (2022) [8]	Systematic Mapping	Literature study	N/A	N/A	Aggregated existing literature; provides no new empirical benchmarking results.
8	Zhou et al. (2021) [9]	LightGBM + SMOTE	NASA-MDP, Jureczko	AUC-ROC, MCC	AUC = 0.81	Retains classical tabular algorithms; over-dependent on synthetic oversampling.
9	Huang et al. (2020) [10]	CNN	PROMISE, NASA-MDP	F1-Score	F1 = 0.68	No explicit compensation mechanisms for severe minority class

Missing Empirical Experiments

- Eg. Predictive Performance Evaluation of Artificial Intelligence Models in Software Defect Prediction: An Empirical Comparison of Machine Learning and Tabular Deep Learning
- Need more experiment, such as:
  - Select exact (some) defect datasets
  - Compare ML alg. Using F1, MCC, …
  - Using Wilcoxon signed-rank test to determine whether performance differences are significant.

Dataset	Random Forest (F1)	TabNet (F1)
Data_1		
Data_2		
…		

Wilcoxon test checks whether TabNet is consistently better than Random Forest across datasets.

Rules:

- If p-value < 0.05: reject H0. The difference is statistically significant.
- If p-value >= 0.05: cannot reject H0. The difference is not statistically significant.

Wilcoxon – p_value

Dataset	Random Forest (F1)	TabNet (F1)	Abs. Diff.	Rank
Data_1	0.42	0.45	0.03	6 → 7.5
Data_2	0.51	0.49	0.02	1 → 3
Data_3	0.47	0.50	0.03	7
Data_4	0.33	0.36	0.03	8
Data_5	0.56	0.58	0.02	2 → 3
Data_6	0.49	0.53	0.04	10
Data_7	0.41	0.43	0.02	3 → 3
Data_8	0.45	0.47	0.02	4 →3
Data_9	0.52	0.55	0.03	9 → 3
Data_10	0.38	0.40	0.02	5 → 3

The smallest absolute difference gets rank 1, the next gets rank 2, and so on.

0.02 average range = (1+2+3+4+5)/5 = 3

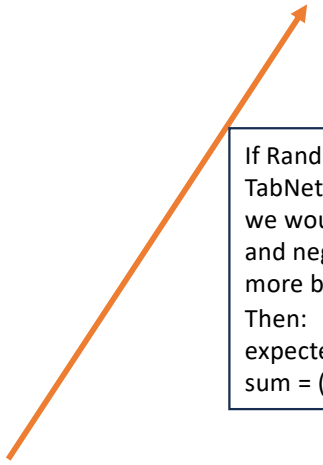
0.03 average range = (6+7+8+9)/4 = 7.5

0.04 average range = (10)/1 = 10

Wilcoxon – p_value

Dataset	Random Forest (F1)	TabNet (F1)	Diff.	Rank
Data_1	0.42	0.45	0.03	7.5
Data_2	0.51	0.49	- 0.02	3
Data_3	0.47	0.50	0.03	7.5
Data_4	0.33	0.36	0.03	7.5
Data_5	0.56	0.58	0.02	3
Data_6	0.49	0.53	0.04	10
Data_7	0.41	0.43	0.02	3
Data_8	0.45	0.47	0.02	3
Data_9	0.52	0.55	0.03	7.5
Data_10	0.38	0.40	0.02	3

Positive differences' rank = W+
= 7.5*4 + 3*4 + 10 = 52
Negative differences' rank = W-
= 3
W = min(W+, W-) = 3



If Random Forest and TabNet were truly equal, we would expect positive and negative ranks to be more balanced.
Then:
expected positive rank sum = (52 + 3) / 2 = 27.5

It is very unbalanced. Almost all rank weight favors TabNet.

Wilcoxon – p_value

Dataset	Random Forest (F1)	TabNet (F1)	Diff.	Rank
Data_1	0.42	0.45	0.03	7.5
Data_2	0.51	0.49	- 0.02	3
Data_3	0.47	0.50	0.03	7.5
Data_4	0.33	0.36	0.03	7.5
Data_5	0.56	0.58	0.02	3
Data_6	0.49	0.53	0.04	10
Data_7	0.41	0.43	0.02	3
Data_8	0.45	0.47	0.02	3
Data_9	0.52	0.55	0.03	7.5
Data_10	0.38	0.40	0.02	3

Smaller rank sum = W- = 3

10 pair → 2^10 possible
All cases where the smaller rank sum is 3 or less?
- 1 rank of 3
Or "no negative ranks"

➔ 5 case "range of 3"
➔ 1 case no negative rank

So, total: 6 cases
Additional, it is two-sided test, we also count the opposite extreme
➔ Total: 6*2 = 12 cases

P_value = 12/2^10 = 0.0117